

Travaux dirigés

Calcul de complexité algorithmique

Cas itératif • Cas récursif

CPGE — filières MP / PSI | Informatique pour tous

Conventions et notations

- n désigne toujours la *taille de l'entrée*. On compte le nombre d'*opérations élémentaires* (affectations, comparaisons, opérations arithmétiques sur des entiers machine), chacune de coût $\Theta(1)$.
- $f = O(g) : \exists C > 0, \exists n_0, \forall n \geq n_0, |f(n)| \leq C g(n)$.
- $f = \Omega(g) : g = O(f)$. $f = \Theta(g) : f = O(g) \text{ et } f = \Omega(g)$.
- $f = o(g) : f(n)/g(n) \rightarrow 0$.
- Sommes usuelles : $\sum_{k=1}^n k = \frac{n(n+1)}{2}$, $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$, $\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1} \ (x \neq 1)$,
- Série harmonique : $H_n = \sum_{k=1}^n \frac{1}{k} = \ln n + \gamma + o(1)$.
- Sauf mention contraire, $\log$ désigne le logarithme en base 2.

Méthode de travail. Pour chaque exercice on demande, dans cet ordre : (1) l'*expression exacte* du nombre d'opérations lorsqu'elle est accessible ; (2) un *équivalent* ou un encadrement ; (3) la classe Θ finale. Répondre « c'est en $O(n^2)$ » sans justification ne vaut aucun point.

Partie I — Le cas itératif : boucles for imbriquées

Exercice 1 — Échauffement — compter avant de majorer

Pour chacun des fragments suivants, donner le nombre *exact* d'exécutions de l'instruction `s += 1`, puis la complexité en Θ .

```

1  # (a)
2  s = 0
3  for i in range(n):
4      s += 1
5
6  # (b)
7  s = 0
8  for i in range(n):
9      for j in range(n):
10         s += 1
11
```

```

12 # (c) m et n sont deux entiers independants
13 s = 0
14 for i in range(n):
15     for j in range(m):
16         s += 1
17
18 # (d)
19 s = 0
20 for i in range(n):
21     for j in range(5):
22         for k in range(n):
23             s += 1

```

Question de synthèse. Dans (c), peut-on dire que la complexité est $\Theta(n^2)$? Discuter selon la relation entre m et n .

Exercice 2 — Boucles triangulaires

Même travail. On donnera à chaque fois la somme exacte *sous forme close*.

```

1 # (a)
2 for i in range(n):
3     for j in range(i):
4         s += 1
5
6 # (b)
7 for i in range(n):
8     for j in range(i, n):
9         s += 1
10
11 # (c)
12 for i in range(1, n+1):
13     for j in range(i, n+1):
14         for k in range(j, n+1):
15             s += 1

```

1. Traiter (a) et (b) et vérifier que les deux sommes diffèrent de n .
2. Pour (c), montrer que le nombre d'itérations est le nombre de triplets $1 \leq i \leq j \leq k \leq n$, c'est-à-dire $\binom{n+2}{3}$. En déduire un équivalent.
3. **Généralisation.** Quelle est la complexité de p boucles imbriquées de ce type ($1 \leq i_1 \leq i_2 \leq \dots \leq i_p \leq n$)?

Exercice 3 — Indices strictement croissants

Soit T un tableau de n entiers.

```

1 # (a) recherche d'un triplet de somme nulle
2 for i in range(n):
3     for j in range(i+1, n):
4         for k in range(j+1, n):
5             if T[i] + T[j] + T[k] == 0:
6                 return True
7 return False
8
9 # (b)

```

```

10 for i in range(n):
11     for j in range(i+1, n):
12         for k in range(n):
13             s += 1

```

1. Pour (a), donner le nombre exact de tests effectués *dans le pire des cas* et la complexité correspondante.
2. Quelle est la complexité de (a) dans le *meilleur* des cas ? Conclusion sur l'écriture $\Theta(\cdot)$ pour cet algorithme.
3. Pour (b), donner le nombre exact d'itérations et la classe Θ . Comparer avec (a) : l'ordre de grandeur change-t-il ?

Exercice 4 — Bornes non linéaires et pas variable

Donner pour chaque fragment le nombre d'itérations sous forme de somme, puis un équivalent et la classe Θ .

```

1 # (a)
2 for i in range(n):
3     for j in range(i*i):
4         s += 1
5
6 # (b)
7 for i in range(1, n+1):
8     for j in range(1, n+1, i):      # pas egal a i
9         s += 1
10
11 # (c)
12 for i in range(1, n+1):
13     j = 0
14     while j*j <= i:
15         j += 1
16         s += 1

```

Pour (b), le nombre d'itérations internes est $\lceil n/i \rceil$. On pensera à la série harmonique. Pour (c), encadrer $\sum_{i=1}^n \sqrt{i}$ par des intégrales.

Exercice 5 — Progression géométrique : attention aux fausses évidences

Les deux fragments ci-dessous ont une boucle externe qui ne fait que $\Theta(\log n)$ tours. Pourtant leurs complexités diffèrent radicalement.

```

1 # (a)
2 i = 1
3 while i < n:
4     for j in range(i):
5         s += 1
6     i = 2*i
7
8 # (b)
9 i = 1
10 while i < n:
11     for j in range(n):

```

```

12     s += 1
13     i = 2*i

```

1. Calculer exactement le nombre d'exécutions de `s += 1` dans (a), en supposant $n = 2^p$. Conclure. Le résultat vous surprend-il ?
2. Même question pour (b).
3. **Moralité.** Énoncer en une phrase la règle qui distingue les deux cas (« le coût total d'une boucle est la *somme* des coûts des tours, pas le produit *nombre de tours* $\times$ *coût du dernier tour* »).
4. Modifier (a) en remplaçant `i = 2*i` par `i = i*i` (avec `i` initialisé à 2). Quelle est alors la complexité ?

Partie II — Le cas itératif : boucles `while`

Méthode pour une boucle `while`

Une boucle `while` n'a pas de nombre de tours écrit dans le code. On procède en trois temps : (i) on identifie un *variant* de boucle, c'est-à-dire une quantité entière positive strictement décroissante à chaque tour (cela prouve la terminaison); (ii) on exprime la valeur de la variable de contrôle *après* k tours; (iii) on résout l'inéquation d'arrêt en k .

Exercice 6 — Les trois schémas fondamentaux

Pour chaque boucle, exprimer la valeur de la variable après k tours, en déduire le nombre exact de tours, puis la complexité en fonction de n ($n \geq 1$ entier).

```

1  # (a)
2  x = n
3  while x > 0:
4      x = x - 1
5
6  # (b)  c > 0 constante entiere
7  x = n
8  while x > 0:
9      x = x - c
10
11 # (c)
12 x = n
13 while x > 1:
14     x = x // 2
15
16 # (d)
17 x = 1
18 while x < n:
19     x = 3*x

```

1. Traiter (a)–(d). Pour (c), montrer que le nombre de tours vaut exactement $\lfloor \log_2 n \rfloor$.
2. Dans (b), la complexité dépend-elle de c ? Que dit-on alors de c vis-à-vis du Θ ?
3. **Piège classique.** Dans (c), l'entrée est l'entier n . Si l'on convient que la taille de l'entrée est le nombre de *bits* de n , soit $t = \lfloor \log_2 n \rfloor + 1$, quelle est la complexité en fonction de t ? Commenter (a) à la lumière de cette convention.

Exercice 7 — Algorithme d'Euclide

```

1 def pgcd(a, b):
2     while b != 0:
3         a, b = b, a % b
4     return a

```

1. Montrer que la suite des seconds arguments est strictement décroissante et positive : en déduire la terminaison (préciser le variant).
2. Montrer que si $a > b > 0$ alors $a \bmod b < a/2$. *Indication* : distinguer $b \leq a/2$ et $b > a/2$.
3. En déduire qu'après deux tours de boucle, le premier argument a été au moins divisé par 2, puis que le nombre de tours est $O(\log \min(a, b))$.
4. **Pire des cas.** On note F_k la suite de Fibonacci ($F_0 = 0, F_1 = 1$). Montrer que l'appel $\text{pgcd}(F_{k+1}, F_k)$ effectue exactement k tours de boucle. En déduire que la borne précédente est atteinte et exprimer le nombre de tours en fonction de $\varphi = \frac{1+\sqrt{5}}{2}$.

Exercice 8 — Exponentiation rapide

```

1 def puissance(x, n):
2     r = 1
3     while n > 0:
4         if n % 2 == 1:
5             r = r * x
6         x = x * x
7         n = n // 2
8     return r

```

1. Prouver la correction en exhibant l'invariant de boucle $r \cdot x^n = x_0^{n_0}$ (où x_0, n_0 sont les valeurs initiales).
2. Donner le nombre exact de tours de boucle pour $n \geq 1$.
3. Soit $\nu(n)$ le nombre de 1 dans l'écriture binaire de n . Montrer que le nombre total de multiplications vaut $\lfloor \log_2 n \rfloor + 1 + \nu(n)$. (On distinguera les élévations au carré et les multiplications accumulées dans r .) Quelle multiplication inutile pourrait-on économiser ?
4. En déduire un encadrement de la complexité, puis la classe Θ dans le pire des cas et dans le meilleur des cas (on précisera pour quels n ils sont atteints).

Exercice 9 — Deux indices qui se croisent

Soit T un tableau *trié par ordre croissant* de n entiers deux à deux distincts et s un entier. L'algorithme suivant cherche deux indices $g < d$ tels que $T[g] + T[d] == s$.

```

1 def paire_de_somme(T, s):
2     g, d = 0, len(T) - 1
3     while g < d:
4         total = T[g] + T[d]
5         if total == s:
6             return (g, d)
7         elif total < s:
8             g = g + 1
9         else:

```

```

10         d = d - 1
11     return None

```

1. Proposer un variant de boucle et prouver la terminaison.
2. Montrer que le nombre de tours est au plus $n - 1$, et que cette borne est atteinte.
3. En déduire la complexité dans le pire des cas. Comparer avec l'algorithme naïf à deux boucles `for` imbriquées.
4. (*Correction, plus difficile.*) Montrer l'invariant : « si une solution (i, j) existe avec $i < j$, alors $g \leq i$ et $j \leq d$ ».

Partie III — Le cas récursif : équation caractéristique

Réurrences linéaires à coefficients constants

Soit (u_n) vérifiant, pour $n \geq p$,

$$u_n = a_1 u_{n-1} + a_2 u_{n-2} + \cdots + a_p u_{n-p} + f(n).$$

1. On résout d'abord l'équation *homogène* ($f \equiv 0$) : on cherche les racines de l'équation *caractéristique* $r^p = a_1 r^{p-1} + \cdots + a_p$. Si $r_1, \dots, r_m$ sont les racines de multiplicités $\mu_1, \dots, \mu_m$, la solution générale homogène est $u_n^{(h)} = \sum_{i=1}^m P_i(n) r_i^n$, avec $\deg P_i < \mu_i$. **2.** On cherche une solution *particulière* de la même forme que f (constante si f constante, polynomiale de degré d si f polynomiale de degré d), en majorant le degré de 1 à chaque fois que la forme essayée est déjà solution de l'homogène. **3.** On somme, puis on détermine les constantes avec les conditions initiales.

Exercice 10 — Tours de Hanoï et variantes

On note $T(n)$ le nombre de déplacements effectués par l'algorithme récursif classique.

```

1 def hanoi(n, depart, arrivee, inter):
2     if n == 0:
3         return
4     hanoi(n-1, depart, inter, arrivee)
5     deplacer(depart, arrivee)          # 1 déplacement
6     hanoi(n-1, inter, arrivee, depart)

```

1. Écrire la relation de récurrence vérifiée par T ainsi que sa condition initiale.
2. Résoudre par la méthode de l'équation caractéristique (homogène + particulière). Vérifier le résultat pour $n = 1, 2, 3$.
3. Résoudre de même les récurrences suivantes (avec $T(0) = 0$) :
 - (a) $T(n) = T(n-1) + n$
 - (b) $T(n) = T(n-1) + n^2$
 - (c) $T(n) = 2T(n-1) + n$
 - (d) $T(n) = 3T(n-1) - 2T(n-2)$, avec $T(0) = 1$, $T(1) = 3$.
4. Pour chacune, donner la classe Θ .

Exercice 11 — Coût de Fibonacci naïf

```

1 def fib(n):
2     if n <= 1:
3         return n
4     return fib(n-1) + fib(n-2)

```

On note $C(n)$ le nombre total d'appels à `fib` engendrés par `fib(n)` (l'appel initial compris).

1. Établir la récurrence vérifiée par C , avec ses deux conditions initiales.
2. Résoudre par équation caractéristique. On notera $\varphi = \frac{1+\sqrt{5}}{2}$ et $\psi = \frac{1-\sqrt{5}}{2}$.
3. Montrer que $C(n) = 2F_{n+1} - 1$ où (F_k) est la suite de Fibonacci.
4. En déduire $C(n) \sim \frac{2}{\sqrt{5}} \varphi^{n+1}$ et conclure : la complexité de `fib` est $\Theta(\varphi^n)$.
5. **Application numérique.** Si une machine effectue 10^9 appels par seconde, estimer le temps de calcul de `fib(80)`.
6. Que devient la complexité si l'on mémorise les appels ? Justifier brièvement.

Exercice 12 — Racine double et second membre

Résoudre complètement, puis donner la classe Θ :

- (a) $T(n) = 4T(n-1) - 4T(n-2)$, $T(0) = 1$, $T(1) = 6$.
- (b) $T(n) = 2T(n-1) - T(n-2) + 2$, $T(0) = 0$, $T(1) = 1$.
- (c) $T(n) = T(n-1) + T(n-2) + 1$, $T(0) = T(1) = 1$.
- (d) $T(n) = 2T(n-1) + 2^n$, $T(0) = 1$. (*Attention : 2 est racine de l'équation caractéristique.*)

Pour (d), chercher une solution particulière sous la forme $\alpha n 2^n$ et non $\alpha 2^n$.

Partie IV — Le cas récursif : récurrences de partition

Récurrence de partition

Un algorithme « diviser pour régner » découpe une instance de taille n en a sous-instances de taille n/b , avec un coût $f(n)$ de découpage et de recombinaison :

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, b > 1.$$

On suppose systématiquement $n = b^k$ pour éviter les parties entières (le résultat asymptotique reste valable dans le cas général). Deux techniques : **déroulement** (on itère la relation jusqu'au cas de base et on somme la contribution de chaque niveau) ou **arbre des appels** (niveau i : a^i nœuds de coût $f(n/b^i)$ chacun).

Exercice 13 — Déroulement à la main — trois schémas de base

Aucun théorème général n'est autorisé dans cet exercice : on déroulera la récurrence. On prendra $n = 2^k$ et $T(1) = 1$.

1. **Recherche dichotomique.** $T(n) = T(n/2) + 1$. Écrire les $k+1$ niveaux, sommer, conclure.
2. **Somme des éléments d'un tableau par dichotomie.**

```

1 def somme(T, g, d):                # somme de T[g..d]
2     if g == d:
3         return T[g]
4     m = (g + d) // 2
5     return somme(T, g, m) + somme(T, m+1, d)

```

Établir $T(n) = 2T(n/2) + \Theta(1)$ et la résoudre par déroulement. Conclure. Cette version est-elle meilleure que la boucle `for` naïve ?

- Coût de recombinaison linéaire.** Résoudre $T(n) = 2T(n/2) + n$ par déroulement : calculer le coût de chaque niveau, le nombre de niveaux, et conclure.
- Minimum et maximum simultanés.** On cherche le couple $(\min, \max)$ d'un tableau de $n = 2^k$ éléments en découpant en deux moitiés. Écrire la récurrence du nombre de *comparaisons* $C(n)$ avec $C(2) = 1$, et montrer que $C(n) = \frac{3n}{2} - 2$. Comparer aux $2n - 3$ comparaisons de l'algorithme itératif naïf.

Exercice 14 — Théorème général (Master Theorem)

Théorème général

Soient $a \geq 1$, $b > 1$ et f une fonction positive. Soit $T(n) = aT(n/b) + f(n)$. On pose $\alpha = \log_b a$. Alors :

- **Cas 1** : si $f(n) = O(n^{\alpha-\varepsilon})$ pour un $\varepsilon > 0$, alors $T(n) = \Theta(n^\alpha)$.
- **Cas 2** : si $f(n) = \Theta(n^\alpha \log^c n)$ avec $c \geq 0$, alors $T(n) = \Theta(n^\alpha \log^{c+1} n)$.
- **Cas 3** : si $f(n) = \Omega(n^{\alpha+\varepsilon})$ pour un $\varepsilon > 0$ et s'il existe $c < 1$ tel que $af(n/b) \leq cf(n)$ pour n assez grand (condition de régularité), alors $T(n) = \Theta(f(n))$.

- Appliquer le théorème général à chacune des récurrences suivantes, en précisant à chaque fois a , b , $\alpha = \log_b a$ et le cas invoqué :
 - $T(n) = 9T(n/3) + n$
 - $T(n) = T(2n/3) + 1$
 - $T(n) = 4T(n/2) + n^2$
 - $T(n) = 3T(n/4) + n \log n$
 - $T(n) = 2T(n/2) + n/\log n$
- Pour celle(s) des récurrences ci-dessus qui *échappe(nt)* au théorème, expliquer précisément pourquoi aucun des trois cas ne s'applique.
- Résoudre $T(n) = 2T(n/2) + n \log n$ par la méthode de l'arbre des appels (le théorème ne s'applique pas directement sous la forme du cas 1 ou 3) et montrer que $T(n) = \Theta(n \log^2 n)$.
- Pourquoi la condition de régularité du cas 3 est-elle nécessaire ? Donner l'idée sur l'exemple $f(n) = n^2 \sin^2 n$ avec $a = b = 2$.

Exercice 15 — Multiplications rapides : gagner un appel récursif

- Produit de deux entiers (Karatsuba).** Soient x et y deux entiers de n chiffres, $n = 2m$. On écrit $x = x_1 10^m + x_0$ et $y = y_1 10^m + y_0$.
 - Écrire xy en fonction de $x_1 y_1$, $x_1 y_0$, $x_0 y_1$, $x_0 y_0$. En déduire la récurrence de l'algorithme naïf et sa complexité.

- (b) Vérifier l'identité $x_1y_0 + x_0y_1 = (x_1 + x_0)(y_1 + y_0) - x_1y_1 - x_0y_0$. Combien de multiplications récursives suffisent alors ?
 - (c) En déduire $T(n) = 3T(n/2) + \Theta(n)$ et résoudre. Donner la valeur numérique de l'exposant $\log_2 3$ à 10^{-3} près.
2. **Produit de matrices (Strassen).** L'algorithme de Strassen multiplie deux matrices $n \times n$ en 7 produits de matrices $n/2 \times n/2$ et $\Theta(n^2)$ additions.
- (a) Écrire la récurrence de l'algorithme par blocs naïf (8 produits) et la résoudre : retrouve-t-on la complexité de l'algorithme usuel ?
 - (b) Résoudre la récurrence de Strassen et comparer.
 - (c) À partir de quelle valeur de a la récurrence $T(n) = aT(n/2) + \Theta(n^2)$ cesse-t-elle d'être en $\Theta(n^2 \log n)$?

Exercice 16 — Changement de variable

Lorsque l'argument récursif n'est pas de la forme n/b , on se ramène au cas standard par un changement de variable.

1. Résoudre $T(n) = T(\sqrt{n}) + 1$, $T(2) = 1$, en posant $n = 2^k$ puis $S(k) = T(2^k)$. Conclure : $T(n) = \Theta(\log \log n)$.
2. Résoudre $T(n) = 2T(\sqrt{n}) + \log_2 n$, $T(2) = 1$, par la même méthode.
3. Résoudre $T(n) = \sqrt{n} T(\sqrt{n}) + n$ (on posera $U(k) = S(k)/2^k$ après le changement de variable).
4. **Application.** Un algorithme sur un ensemble d'entiers de $\{1, \dots, n\}$ se ramène récursivement à un sous-problème sur $\{1, \dots, \sqrt{n}\}$ avec un coût de recombinaison $\Theta(n)$. Écrire sa récurrence et donner sa complexité.

Fin de l'énoncé — 16 exercices.